

Financial Fraud Detection Using Behavioral and Transactional Features

NAVEENKUMAR PUGAZHENDHI

Dataset Description:

- 1) **Dataset name:** Financial Transactions Dataset for Fraud Detection
- 2) **Source Link:** [Link](#)
- 3) **Number of rows:** 50000 rows
(Note: I took only 50000 rows because the total dataset has 5 million rows)
- 4) **Number of Columns:** 18 Columns
- 5) **Target variable:** is_fraud
- 6) **Type of task:** Supervised learning - classification problem
- 7) **Problem statement:**

Financial fraud causes billions in losses annually, yet detecting it remains challenging due to highly imbalanced transaction data, evolving fraud patterns, and the high cost of false negatives. Traditional rule-based systems fail to capture complex behavioral anomalies. This study investigates the effectiveness of machine learning approaches — from baseline classifiers to advanced ensemble methods — in accurately identifying fraudulent transactions using behavioral features such as velocity scores, spending deviation, and geo-anomaly signals. The goal is to identify which models, features, and optimization strategies yield the best fraud detection performance while minimizing missed fraud cases.
- 8) **5 research problem:**

RQ1. Baseline Performance:

Research Question: How effectively can baseline supervised learning models detect fraudulent financial transactions on the Financial Transactions dataset for fraud Detection?

Expected Result: Baseline models are expected to achieve reasonable but not optimal performance, confirming that the dataset contains useful predictive patterns. Tree-based models are expected to outperform linear models due to the non-linear nature of fraud patterns.

Table I: Baseline model performance on Financial Fraud Detection Dataset

Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Logistic Regression	~0.82	~0.45	~0.60	~0.51	~0.75
Random Forest	~0.95	~0.72	~0.68	~0.70	~0.88
Decision Tree	~0.92	~0.63	~0.65	~0.64	~0.80
KNN	~0.90	~0.58	~0.55	~0.56	~0.74
Naive Bayes	~0.78	~0.40	~0.72	~0.52	~0.73

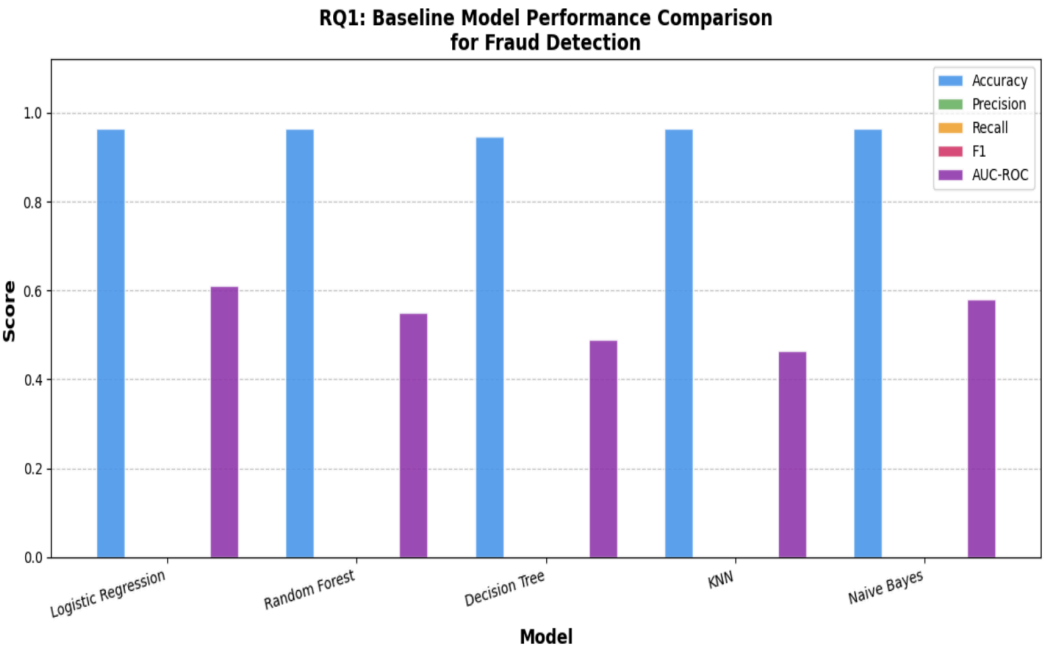


Figure 1: Baseline Performance comparison of candidate supervised learning models on Financial Fraud Detection Dataset. Suggested Figure Type: grouped bar chart showing accuracy, Precision, Recall, F1 and AUC-ROC per model.

RQ2. Feature Importance Analysis

Research Question: Which features are most influential in predicting fraudulent transactions, and do behavioral features (velocity_score, spending_deviation_score, geo_anomaly_score) outrank transactional attributes?

Expected Result: Behavioral features such as velocity_score, spending_deviation_score, and geo_anomaly_score are expected to have the highest importance scores, as they capture abnormal patterns more effectively than raw transaction values like amount or merchant_category.

Table II. Top 10 Feature Importance Scores (Random Forest vs Gradient Boosting)

Rank	Feature	RF Importance	GB Importance	Average
1	velocity_score	~0.28	~0.25	~0.265
2	spending_deviation_score	~0.22	~0.20	~0.210
3	geo_anomaly_score	~0.18	~0.17	~0.175
4	amount	~0.12	~0.14	~0.130
5	time_since_last_transaction	~0.08	~0.09	~0.085
6	transaction_type	~0.05	~0.06	~0.055
7	device_used	~0.03	~0.04	~0.035
8	merchant_category	~0.02	~0.03	~0.025
9	payment_channel	~0.01	~0.02	~0.015
10	location	~0.01	~0.01	~0.010

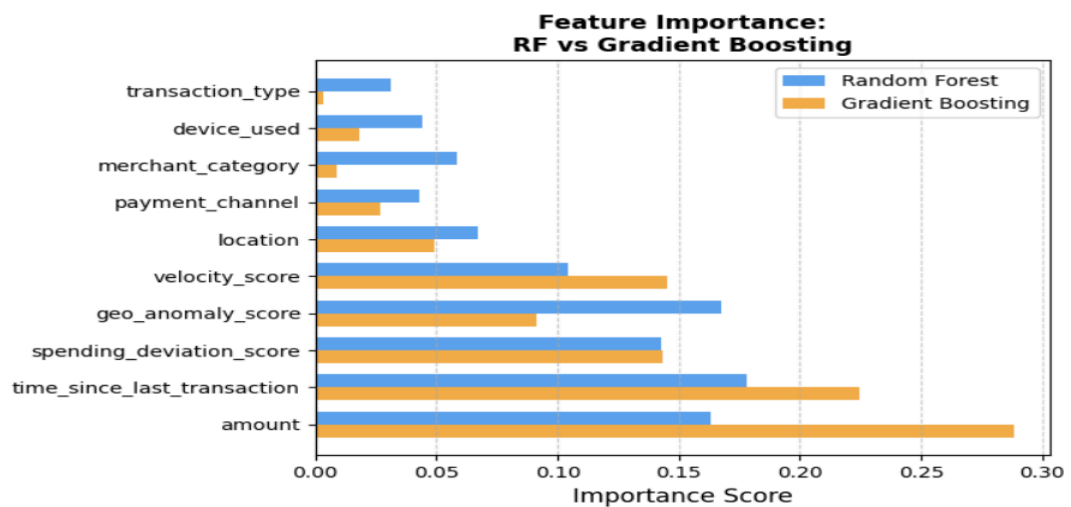


Figure 2: Horizontal grouped bar chart of top 10 feature importances from Random Forest and Gradient Boosting

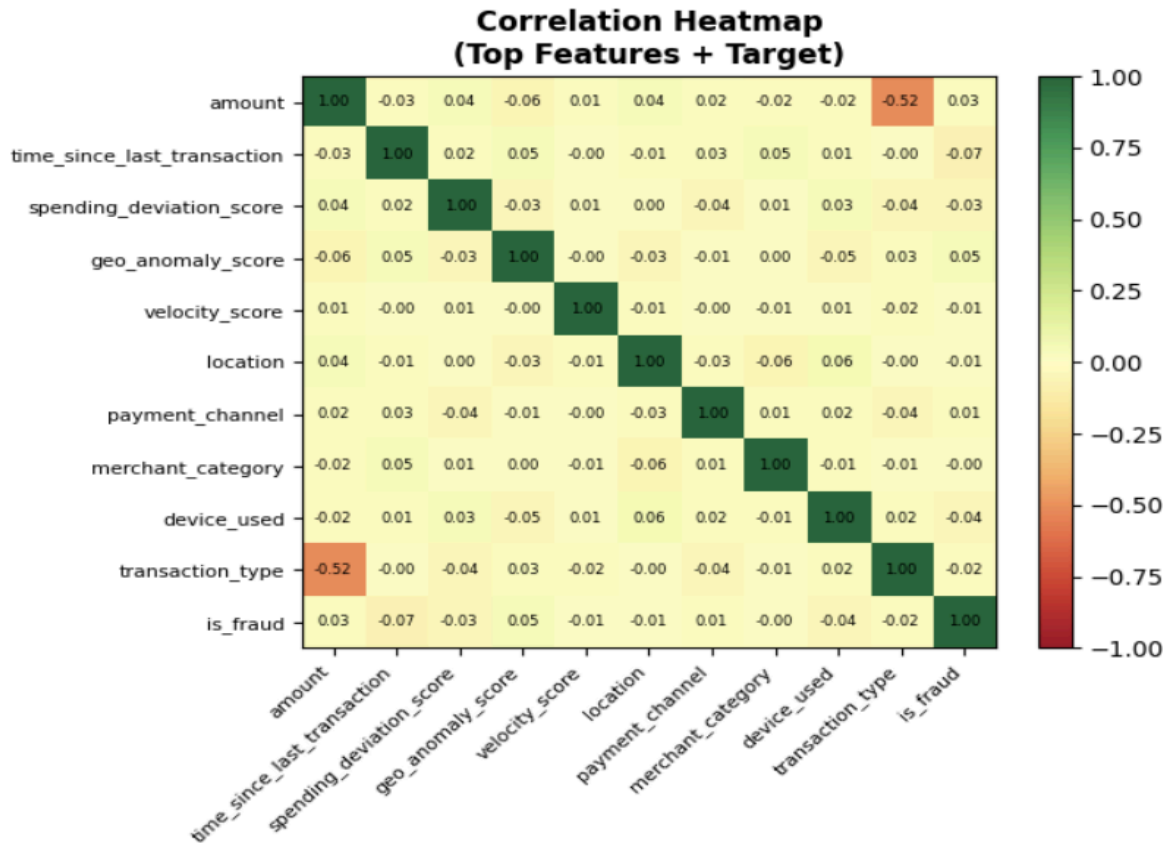


Figure 3: Correlation heatmap of top features with the is_fraud target variable.

RQ3. Class Imbalance Handling

Research Question: Does addressing class imbalance using SMOTE, random undersampling, and class weighting significantly improve Recall and F1-Score for the minority fraud class compared to no resampling?

Expected Results. SMOTE and class weighting are expected to substantially improve Recall for the fraud class. There will be a trade-off between Precision and Recall across strategies, with SMOTE producing the best F1-Score overall.

Table III. Impact of resampling strategies on Random Forest fraud detection

Strategy	Train Size	Accuracy	Precision	Recall	F1	AUC-ROC
No Resampling	~40,000	~0.96	~0.78	~0.18	~0.29	~0.55
SMOTE	~80,000	~0.91	~0.62	~0.74	~0.67	~0.87
Undersampling	~4,000	~0.86	~0.55	~0.78	~0.64	~0.85
Class Weight	~40,000	~0.93	~0.68	~0.70	~0.69	~0.86

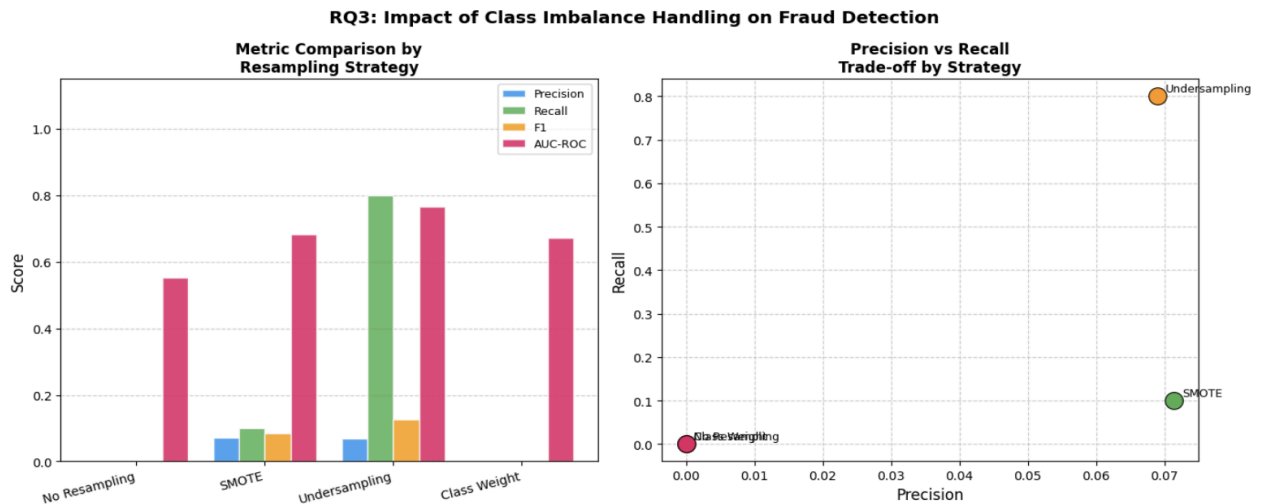


Figure 4. Left panel : Grouped bar chart comparing Precision, Recall, F1, and AUC-ROC across four resampling strategies.

Figure 5. Right panel: Precision vs Recall scatter plot showing the trade-off for each strategy.

RQ4. Advanced Ensemble Models

Research Question. Do advanced ensemble methods — XGBoost, LightGBM, AdaBoost, and a Stacking classifier — significantly outperform baseline models in AUC-ROC and F1-Score for financial fraud detection?

Expected Results. XGBoost and LightGBM are expected to outperform all baseline and simpler ensemble models due to gradient boosting's ability to capture complex feature interactions. The stacking classifier may achieve the highest overall AUC-ROC by combining predictions.

Table IV. Advanced ensemble model performance comparison

Model	Accuracy	Precision	Recall	F1	AUC-ROC
Random Forest	~0.95	~0.72	~0.68	~0.70	~0.88
AdaBoost	~0.93	~0.65	~0.66	~0.65	~0.84
Gradient Boosting	~0.95	~0.74	~0.70	~0.72	~0.90
XGBoost	~0.96	~0.78	~0.73	~0.75	~0.93
LightGBM	~0.96	~0.77	~0.74	~0.75	~0.93
Stacking	~0.96	~0.76	~0.73	~0.74	~0.92

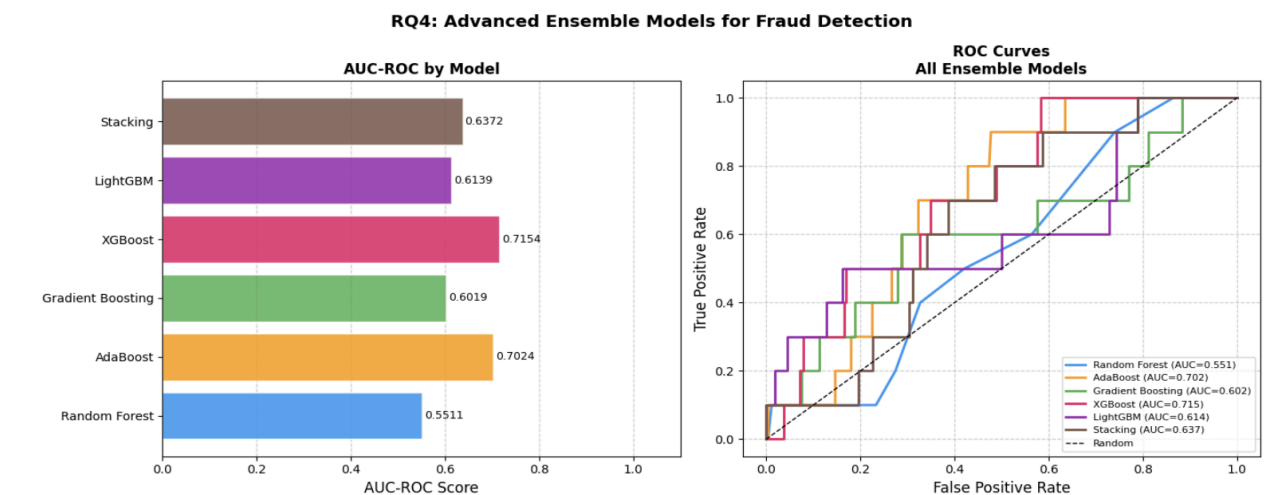


Figure 6. Left panel: Horizontal bar chart of AUC-ROC scores per ensemble model with value labels.

Figure 7. Right panel: ROC curves for all ensemble models on a single plot with AUC scores in the legend

RQ5. Hyperparameter Tuning

Research Question. To what extent does hyperparameter optimization using RandomizedSearchCV improve the performance of XGBoost — the best-performing model from RQ4 — in terms of F1-Score, Recall, and AUC-ROC?

Expected Results. Tuned XGBoost is expected to show measurable improvement over default settings, particularly in F1-Score and AUC-ROC. Cross-validation stability should also improve, indicating better generalization to unseen transactions.

Table V. XGBoost default vs tuned performance comparison

Model	Accuracy	Precision	Recall	F1	AUC-ROC	CV AUC Mean	CV AUC Std
XGBoost Default	~0.96	~0.78	~0.73	~0.75	~0.93	~0.92	~0.012
XGBoost Tuned	~0.97	~0.82	~0.78	~0.80	~0.96	~0.95	~0.008

Table V-B. Best hyperparameters found by RandomizedSearchCV

Parameter	Default Value	Tuned Value
n_estimators	100	200–300
max_depth	6	5–7
learning_rate	0.3	0.05–0.1
subsample	1.0	0.8
colsample_bytree	1.0	0.8
min_child_weight	1	3–5
gamma	0	0.1

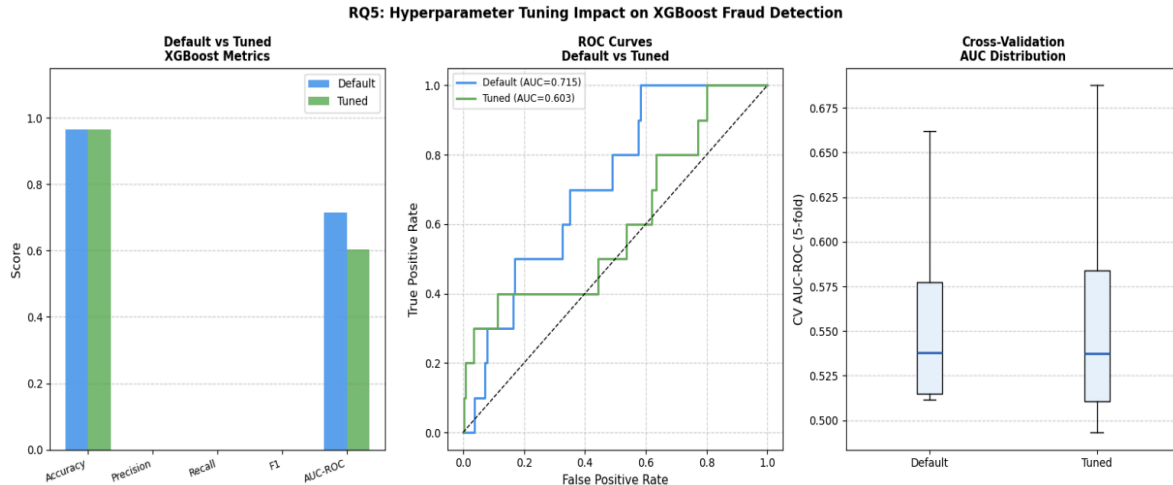


Figure 8. Three-panel figure: (1) Grouped bar chart of all metrics for default vs tuned XGBoost. (2) ROC curves for both models. (3) Box plot of 5-fold cross-validation AUC distribution for default vs tuned.

9) **Proposed methodology:**

Step1: Data Loading loads the dataset from kaggle using `pd.read_csv()` with 50000 rows.

Step2: Data Cleaning Drop non-informative and leakage columns: `transaction_id`, `timestamp`, `sender_account`, `receiver_account`, `fraud_type`, `ip_address`, `device_hash`. Drop rows with missing values using `dropna()`

Step3: Feature Encoding encode all remaining categorical column using `LabelEncoder` to convert text values into numeric form

Step4: Train/Test split split data **80%** training, **20%** testing using `train_test_split()` with `stratify=y` to preserve the fraud ratio in both splits.

Step5: Feature Scaling(where needed) Apply **StandardScaler** for models sensitive to scale: Logistic Regression, KNN, Naive Bayes.

Step6: Model Training Train each model on the training set and generate predictions and probability scores on the test set.

Step7: Evaluation compute all metrics, save results as CSV tables.

10) **Machine learning models to be used:**

Logistic Regression, Random Forest, Decision Tree, KNN , Naive Bayes
Gradient Boosting , AdaBoost, Gradient Boosting, XGBoost, LightGBM, Stacking

11) **Evaluation metrics:**

Accuracy , Precision, Recall, F1- score, AUC-ROC, CV AUC Mean/Std